ETS

PEMROGRAMAN iMCLab BERBASIS BLYNK



Dosen Pengampu:
Dr. Basuki Rahmat, S.Si. MT

Disusun oleh:
Moh Avin Dharma Wijaya (23081010243)

ROBOTIKA A – 081

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"

JAWA TIMUR

2026

DAFTAR ISI

BAB I KONFIGURASI DAN KODE PROGRAM 1

1.1 Konfigurasi template dan datastream Blynk 3

1.2 Daftar perintah serial 4

1.3 Kode program 4

BAB II HASIL DAN PEMBAHASAN 13

2.1 Prosedur unggah program Blynk ke ESP32 14

2.2 Penyambungan perangkat ke server Blynk 15

2.3 Pengujian kendali motor melalui sakelar dashboard 16

2.4 Pengujian pengaturan kecepatan melalui slider 17

2.5 Pengujian pembacaan putaran motor 19

2.6 Keadaan dashboard setelah motor dimatikan 20

2.7 Rangkuman hasil pengujian 21

2.8 Dokumentasi Praktikum 22

BAB III PENUTUP 25

3.1 Kesimpulan 26

3.2 Saran 26

DAFTAR GAMBAR

Gambar 1. Prosedur unggah program Blynk ke board ESP32 14
Gambar 2. Dashboard Blynk saat perangkat berstatus online 15
Gambar 3. Diagram rangkaian perangkat saat dioperasikan melalui dashboard 16
Gambar 4. Kenaikan kecepatan melalui slider menjadi PWM 207 17
Gambar 5. Penurunan kecepatan melalui slider menjadi PWM 195 18
Gambar 6. Keadaan dashboard setelah sakelar dimatikan 20
Gambar 7. Kit iMCLab saat kendali motor dipindahkan ke Blynk 23
Gambar 8. Kit iMCLab dan roda piringan saat dioperasikan dari dashboard Blynk 23
Gambar 9. Dashboard Blynk dengan sakelar, slider, dan gauge kecepatan 24
Gambar 10. Daftar datastream V0 sampai V3 pada Template Blynk 24

BAB I KONFIGURASI DAN KODE PROGRAM

Tujuan modul ketiga ini adalah memindahkan kendali motor DC kit iMCLab dari jalur kabel menuju jaringan internet. Bila dua modul sebelumnya menuntut kehadiran operator di depan Serial Monitor atau notebook Python, modul ini membebaskan operator dari ikatan kabel: selama board dan penelusur web sama-sama terjangkau internet, kendali tetap dapat dilakukan. Perpindahan medium kendali inilah yang menuntut penambahan lapisan perangkat lunak berupa pustaka WiFi dan pustaka Blynk pada firmware, berikut sejumlah datastream sebagai jalur pertukaran data.

Perangkat keras yang dipakai tetap kit iMCLab dengan board DOIT ESP32 DEVKIT V1, dijalankan pada kecepatan serial 115200 baud. Pemetaan pin tidak berubah, yakni LED indikator pada GPIO 2 (`ledPin`), driver motor IN1 pada GPIO 27 (`motor1Pin1`), IN2 pada GPIO 26 (`motor1Pin2`), EN/PWM pada GPIO 12 (`enable1Pin`), dan sensor RPM pada GPIO 13 (`pin_rpm`).

1.1 Konfigurasi template dan datastream Blynk

Sebelum firmware dapat bertukar data, sebuah template pada Blynk Console harus sudah berdiri. Template ini diberi nama `robot`, dan identitasnya diwakili oleh `BLYNK_TEMPLATE_ID` bersama `BLYNK_TEMPLATE_NAME` yang sepadan, sementara perangkat memperkenalkan diri menggunakan token autentikasi pada `BLYNK_AUTH_TOKEN`. Ketiga nilai itu disalin apa adanya pada baris `#define` paling atas kode, sebab satu karakter saja yang melenceng akan membuat perangkat ditolak server.

Jalur data di dalam template diwujudkan sebagai empat datastream virtual pin seperti pada tabel berikut.

Virtual pin	Fungsi	Arah data	Keterangan
V0	RPM	ESP32 → Blynk	Hasil pembacaan sensor encoder, dikirim tiap 1 detik
V1	Nilai duty	ESP32 → Blynk	Nilai duty cycle yang sedang aktif (0–255)
V2	Slider PWM	dua arah	Masukan pengguna untuk mengatur duty cycle
V3	Sakelar ON/OFF	dua arah	Kontrol hidup-mati motor

Dua datastream terakhir bekerja dua arah karena dashboard boleh mengirim perintah ke perangkat, sedangkan V0 dan V1 murni sebagai kanal pelaporan. Perintah dari slider ditangani `BLYNK_WRITE(V2)`, perintah dari sakelar ditangani `BLYNK_WRITE(V3)`, dan laporan balik ke dashboard dijalankan `kirimKeBlynk()` serta `applyMotor()`. Ketika perangkat baru saja pulih dari

putus sambungan, `BLYNK_CONNECTED()` memanggil `Blynk.syncVirtual(V2)` dan `Blynk.syncVirtual(V3)` supaya nilai terakhir pada dua widget kendali itu ditarik kembali dari server.

1.2 Daftar perintah serial

Kendali melalui dashboard bukan satu-satunya jalan masuk perintah. Program masih menyediakan jalur Serial Monitor sebagai cadangan ketika jaringan sedang bermasalah atau ketika operator ingin memeriksa nilai mentah di luar tampilan dashboard.

No	Perintah	Fungsi
1	1	Menyalakan motor; duty cycle dipertahankan, atau disetel 200 bila sebelumnya nol
2	0	Mematikan motor sehingga duty cycle menjadi 0
3	+	Menaikkan duty cycle sebesar 10 satuan (DUTY_STEP)
4	-	Menurunkan duty cycle sebesar 10 satuan (DUTY_STEP)
5	PWM:xxx	Menetapkan duty cycle secara langsung pada rentang 0–255
6	R	Menampilkan nilai RPM dan PWM terkini

1.3 Kode program

Kode program `imclab_Blynk_kelas.ino` berikut disalin utuh, kecuali baris kredensial yang telah disamarkan demi keamanan.

```
/*****************************************************
 * iMCLab + ESP32 + Blynk IoT
```

```
 * RPM Monitoring dan Kontrol PWM
 *****************************************************/

#define BLYNK_TEMPLATE_ID "<TEMPLATE_ID>"
#define BLYNK_TEMPLATE_NAME "robot"
#define BLYNK_AUTH_TOKEN "<AUTH_TOKEN>"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>

//char ssid[] = "<NAMA_WIFI>";
//char pass[] = "<KATA_SANDI_WIFI>";

//Konfigurasi Wifi
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "<NAMA_WIFI>";
char pass[] = "<KATA_SANDI_WIFI>";



// ============================================================
// Motor dan LED
// ============================================================

int ledPin = 2;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// ============================================================
// Sensor RPM
// ============================================================

const byte pin_rpm = 13;
const int HOLES = 2;

volatile unsigned int rev = 0;
volatile unsigned long lastPulseTime = 0;

const unsigned long DEBOUNCE_US = 500;

// ============================================================
// PWM
// ============================================================

const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8;

int dutyCycle = 200;
```

```
const int DUTY_STEP = 10;
const int DUTY_MIN = 0;
const int DUTY_MAX = 255;

bool motorEnable = false;

// ============================================================
// RPM
// ============================================================

float rpm = 0.0;
unsigned long old_time = 0;

// ============================================================

String inputString = "";

BlynkTimer timer;

// ============================================================
// ISR Encoder
// ============================================================

void IRAM_ATTR isr()
{
  unsigned long now = micros();

  if (now - lastPulseTime > DEBOUNCE_US)
  {
    rev++;
    lastPulseTime = now;
  }
}

// ============================================================
// Sync setelah reconnect Blynk
// ============================================================

BLYNK_CONNECTED()
{
  Blynk.syncVirtual(V2);
  Blynk.syncVirtual(V3);
}

// ============================================================
// Hitung RPM
// ============================================================

void hitungRPM()
{
  unsigned long cur_time = millis();
```

```
  if (cur_time - old_time >= 1000)
  {
    detachInterrupt(digitalPinToInterrupt(pin_rpm));

    float rot_per_sec = (float)rev / (float)HOLES;

    rpm = rot_per_sec * 60.0;

    rev = 0;

    attachInterrupt(
      digitalPinToInterrupt(pin_rpm),
      isr,
      FALLING);

    old_time = millis();
  }
}

// ============================================================
// Terapkan PWM ke Motor
// ============================================================

void applyMotor()
{
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, dutyCycle);

  digitalWrite(ledPin, dutyCycle > 0);

  // Sinkronisasi ke Blynk
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Status Serial
// ============================================================

void tampilkanStatus(const char* aksi)
{
  Serial.print(aksi);
  Serial.print(" | PWM=");
  Serial.print(dutyCycle);
  Serial.print(" | RPM=");
  Serial.println(rpm, 1);
}
```

```
// ============================================================
// Kirim Data ke Blynk
// ============================================================

void kirimKeBlynk()
{
  Blynk.virtualWrite(V0, rpm);
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// ============================================================
// Slider PWM dari Blynk (V2)
// ============================================================

BLYNK_WRITE(V2)
{
  dutyCycle = constrain(
    param.asInt(),
    DUTY_MIN,
    DUTY_MAX);

  motorEnable = (dutyCycle > 0);

  applyMotor();

  Serial.print("PWM dari Blynk = ");
  Serial.println(dutyCycle);
}

// ============================================================
// Tombol ON/OFF dari Blynk (V3)
// ============================================================

BLYNK_WRITE(V3)
{
  motorEnable = param.asInt();

  if (motorEnable)
  {
    if (dutyCycle == 0)
      dutyCycle = 200;
  }
  else
  {
    dutyCycle = 0;
  }

  applyMotor();
```

```
  Serial.print("Motor ");
  Serial.println(motorEnable ? "ON" : "OFF");
}

// ============================================================
// Setup
// ============================================================

void setup()
{
  Serial.begin(115200);

  pinMode(ledPin, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);

  pinMode(pin_rpm, INPUT_PULLUP);

  attachInterrupt(
    digitalPinToInterrupt(pin_rpm),
    isr,
    FALLING);

  ledcSetup(
    pwmChannel,
    freq,
    resolution);

  ledcAttachPin(
    enable1Pin,
    pwmChannel);

  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);

  ledcWrite(pwmChannel, 0);

  old_time = millis();

//  Blynk.begin(
//    BLYNK_AUTH_TOKEN,
//    ssid,
//    pass);

  // Koneksi ke Blynk
  Blynk.begin(auth, ssid, pass);

  timer.setInterval(
    1000L,
```

```
    kirimKeBlynk);

  Serial.println("================================");
  Serial.println("iMCLab + Blynk Ready");
  Serial.println("================================");
  Serial.println("1 = ON");
  Serial.println("0 = OFF");
  Serial.println("+ = PWM naik");
  Serial.println("- = PWM turun");
  Serial.println("PWM:200");
  Serial.println("R = lihat RPM");
}

// ============================================================
// Loop
// ============================================================

void loop()
{
  Blynk.run();
  timer.run();

  hitungRPM();

  while (Serial.available() > 0)
  {
    char c = Serial.read();

    if (c == '\n' || c == '\r')
    {
      inputString.trim();

      if (inputString.startsWith("PWM:"))
      {
        int val = inputString.substring(4).toInt();

        val = constrain(
          val,
          DUTY_MIN,
          DUTY_MAX);

        dutyCycle = val;
        motorEnable = (dutyCycle > 0);

        applyMotor();

        tampilkanStatus("Set PWM");
      }

      else if (inputString.length() > 0)
      {
        char cmd = inputString.charAt(0);
```

```
if (cmd == '1')
{
  if (dutyCycle == 0)
    dutyCycle = 200;

  motorEnable = true;

  applyMotor();

  tampilkanStatus("Motor ON");
}

else if (cmd == '0')
{
  dutyCycle = 0;
  motorEnable = false;

  applyMotor();

  Serial.println("Motor OFF");
}

else if (cmd == '+')
{
  dutyCycle += DUTY_STEP;

  if (dutyCycle > DUTY_MAX)
    dutyCycle = DUTY_MAX;

  motorEnable = true;

  applyMotor();

  tampilkanStatus("PWM NAIK");
}

else if (cmd == '-')
{
  dutyCycle -= DUTY_STEP;

  if (dutyCycle < DUTY_MIN)
    dutyCycle = DUTY_MIN;

  motorEnable = (dutyCycle > 0);

  applyMotor();

  tampilkanStatus("PWM TURUN");
}
```

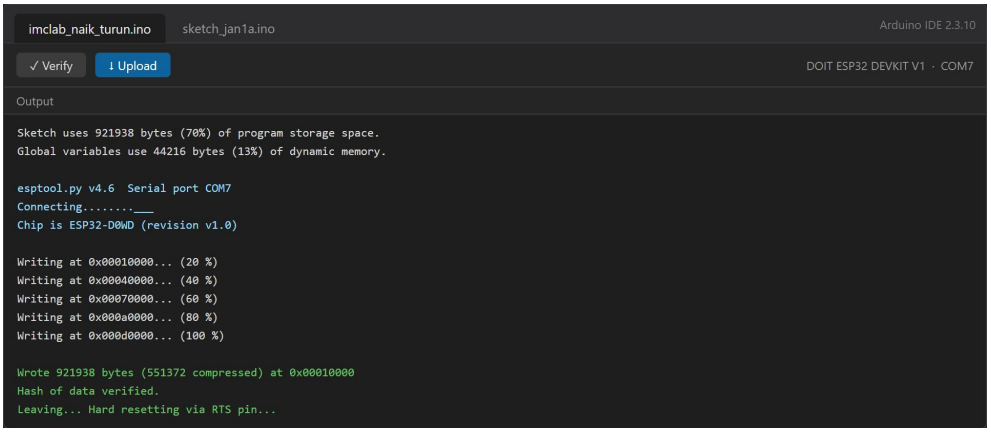
```
        else if (cmd == 'R' || cmd == 'r')
        {
          Serial.print("RPM = ");
          Serial.print(rpm, 1);
          Serial.print(" | PWM = ");
          Serial.println(dutyCycle);
        }
      }

      inputString = "";
    }
    else
    {
      inputString += c;
    }
  }
}
```

BAB II HASIL DAN PEMBAHASAN

2.1 Prosedur unggah program Blynk ke ESP32

Keberhasilan modul ini bergantung pada kesiapan dua sisi yang berbeda, yaitu sisi rancangan di Blynk Console dan sisi firmware di board. Karena itu langkah pertama bukan menekan tombol unggah, melainkan memastikan pustaka Blynk beserta dependensinya sudah terpasang melalui Library Manager, kemudian menyesuaikan `BLYNK_TEMPLATE_ID`, `BLYNK_TEMPLATE_NAME`, `BLYNK_AUTH_TOKEN`, serta nama dan kata sandi WiFi pada berkas kode. Baru setelah kedua sisi itu selaras, proses kompilasi dan unggah dijalankan.



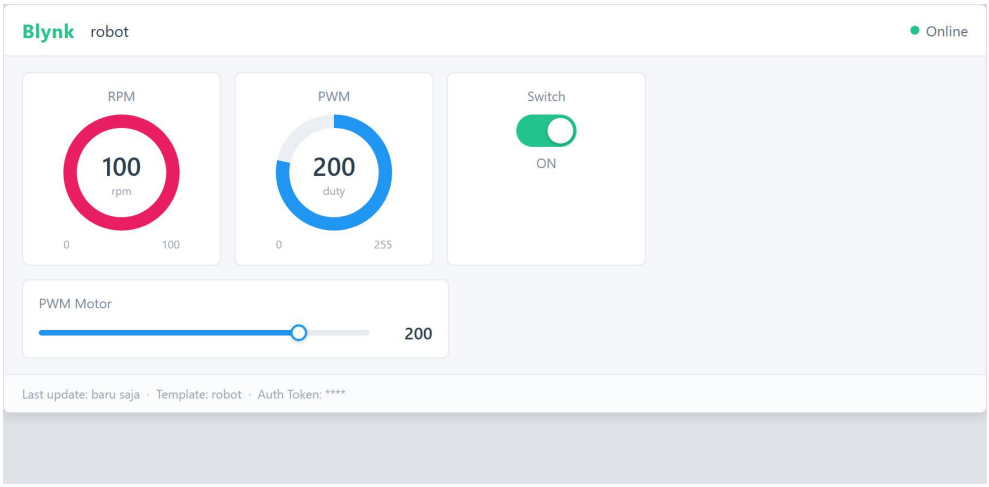
Gambar 1. Prosedur unggah program Blynk ke board ESP32

Jendela Output melaporkan pemakaian memori program sebesar 921.938 byte, atau sekitar 70% dari kapasitas yang tersedia. Angka ini jauh lebih besar dibandingkan program pada modul-modul sebelumnya karena kini ada tambahan pustaka WiFi dan pustaka Blynk yang harus ditautkan ke firmware. Proses penulisan flash diakhiri pesan "Hash of data verified" dan "Hard resetting via RTS pin", dua penanda bahwa unggahan tuntas dan board segera melakukan restart

sendiri.

2.2 Penyambungan perangkat ke server Blynk

Yang diharapkan dari tahap ini adalah berdirinya satu sesi sambungan tetap antara board dan Blynk Cloud. Sesaat setelah restart, ESP32 mencari jaringan WiFi yang namanya telah dicantumkan, memperoleh alamat IP, lalu mendaftarkan diri ke server Blynk Cloud region SGP1 dengan membawa token autentikasi yang tadi ditanam pada kode. Bila token dan Template ID cocok, server menerima perangkat dan menandainya sebagai terhubung.



Gambar 2. Dashboard Blynk saat perangkat berstatus online

Penanda hijau bertuliskan Online di samping nama device `robot` menandakan pendaftaran berhasil. Kondisi awal widget pun sekaligus terlihat di sini:

widget PWM dan slider PWM Motor menampilkan nilai 200, sementara sakelar berada

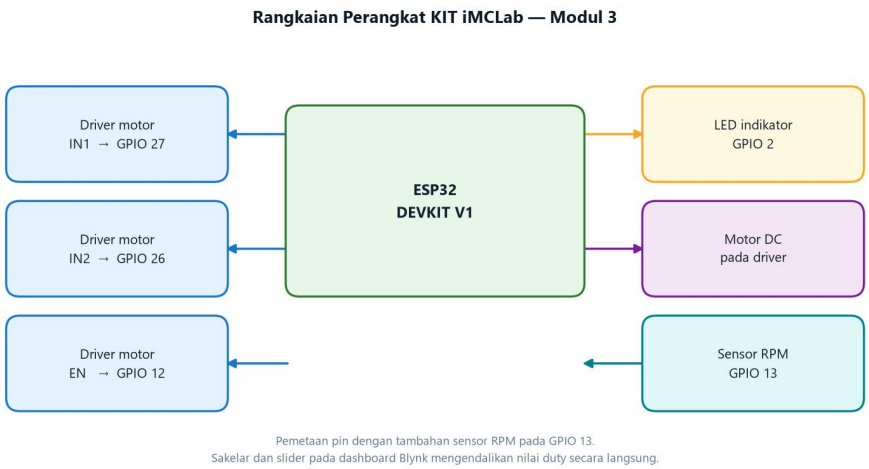
pada posisi aktif. Nilai 200 itu berasal dari nilai awal `dutyCycle`, lalu

diunggah ke dashboard oleh `kirimKeBlynk()` yang dijalankan BlynkTimer setiap

satu detik. Jadi dashboard sudah menampilkan keadaan sebenarnya sejak detik pertama tersambung, tanpa perlu ada widget yang digerakkan lebih dulu.

2.3 Pengujian kendali motor melalui sakelar dashboard

Pengujian ini bertujuan memastikan bahwa satu ketukan pada sakelar dashboard sanggup mengubah keadaan fisik perangkat, sekaligus memeriksa apakah jalur perintah dari penelusur web sampai ke driver motor bekerja tanpa hambatan berarti.



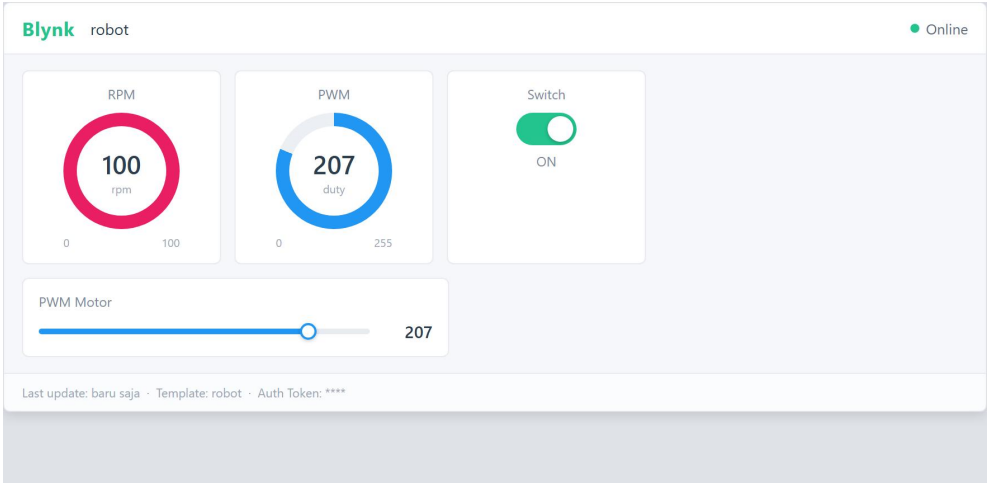
Gambar 3. Diagram rangkaian perangkat saat dioperasikan melalui dashboard

Dalam keadaan sakelar ON, dashboard menampilkan PWM 200 dengan sakelar aktif, sedangkan kit iMCLab memperlihatkan piringan motor yang berputar hingga bayangannya kabur dan LED indikator biru yang menyala. Perintah dari penelusur web tidak langsung sampai ke board, melainkan melintas lebih dulu melalui server Blynk Cloud; meski demikian jeda yang terjadi hanya berada pada orde puluhan milidetik sehingga nyaris tidak dapat dirasakan.

Ada satu perilaku kode yang perlu dicatat pada bagian ini. Variabel `motorEnable` hanya diperbarui dari sisi Serial untuk perintah `1`, `0`, dan `-`, sedangkan perintah `+` menyetelnya `true` tanpa memeriksa keadaan sakelar di dashboard. Akibatnya perintah Serial hanya mengubah status ke satu arah, dan status di dashboard baru benar-benar selaras setelah `BLYNK_CONNECTED()` memanggil `syncVirtual(V2)` dan `syncVirtual(V3)` untuk menarik kembali nilai terakhir yang tersimpan di server.

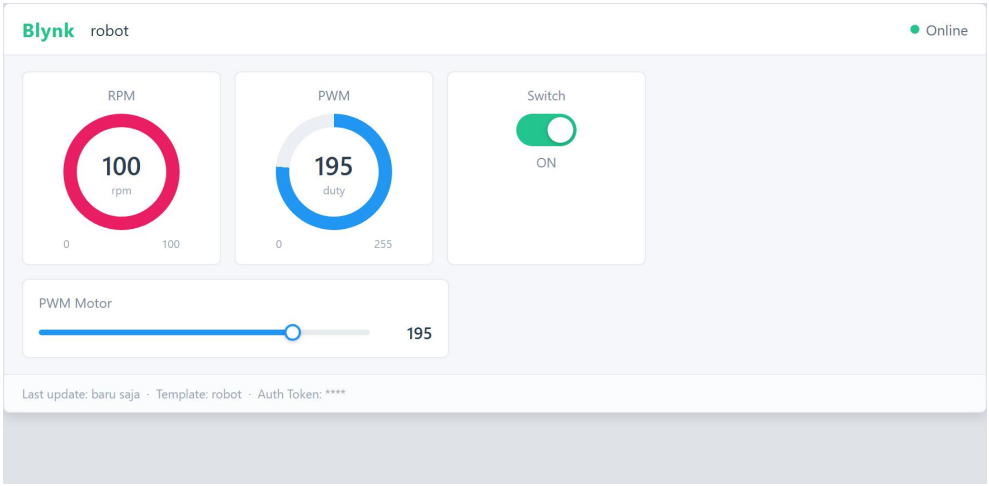
2.4 Pengujian pengaturan kecepatan melalui slider

Fokus pengujian ini adalah ketelitian pengaturan kecepatan: apakah slider mampu memilih nilai duty cycle secara bebas, dan apakah nilai itu benar-benar merepresentasikan daya rata-rata yang diterima motor. Mekanismenya, ketika slider digeser, dashboard mengirim angka ke V2, lalu `BLYNK_WRITE(V2)` membatasi nilai tersebut dengan `constrain()` pada rentang `DUTY_MIN` sampai `DUTY_MAX`, dan `applyMotor()` menuliskannya ke kanal PWM.



Gambar 4. Kenaikan kecepatan melalui slider menjadi PWM 207

Ketika slider digeser ke kanan, dashboard mencatat PWM 207 dan widget PWM ikut menyesuaikan ke angka yang sama. Putaran motor bertambah cepat seiring bertambahnya daya rata-rata yang masuk. Keunggulan slider dibanding perintah + terletak pada kebebasan memilih nilai: perintah + hanya menambah sepuluh satuan tiap penekanan, sedangkan slider dapat melompat ke angka berapa pun dalam satu gerakan.



Gambar 5. Penurunan kecepatan melalui slider menjadi PWM 195

Menggeser slider ke arah kiri menghasilkan duty cycle 195 dan widget PWM kembali menyesuaikan diri, sementara putaran motor melambat. Pada kedua arah gerakan itu tidak pernah muncul angka yang berbeda antara posisi slider dan widget PWM, karena program menuliskan nilai duty cycle terbaru ke V2 setiap kali terjadi perubahan.

Perlu diperhatikan bahwa `applyMotor()` menulis ke tiga virtual pin sekaligus (V1, V2, dan V3) setiap kali dipanggil, sementara `kirimKeBlynk()` juga menulis ke pin yang sama tiap detik. Penulisan ganda ini membuat dashboard

selalu dalam keadaan segar, namun konsekuensinya lalu lintas data ke server Blynk menjadi lebih banyak daripada yang sebenarnya diperlukan.

2.5 Pengujian pembacaan putaran motor

Kemampuan baru pada modul ini adalah pemantauan putaran motor. Nilai RPM disusun di dalam `hitungRPM()` dengan membagi jumlah pulsa encoder (`rev`) dengan jumlah celah piringan (`HOLES = 2`), kemudian dikalikan 60 untuk memperoleh putaran per menit sesuai rumus `rpm = (rev / HOLES) * 60`. Pulsa dihitung oleh rutin `isr()` yang dipicu oleh mode `FALLING` pada pin 13, dengan penapis `DEBOUNCE_US` sebesar 500 mikrodetik agar derau tidak ikut terhitung sebagai pulsa. Hasil terakhir dilaporkan melalui `kirimKeBlynk()` setiap satu detik.

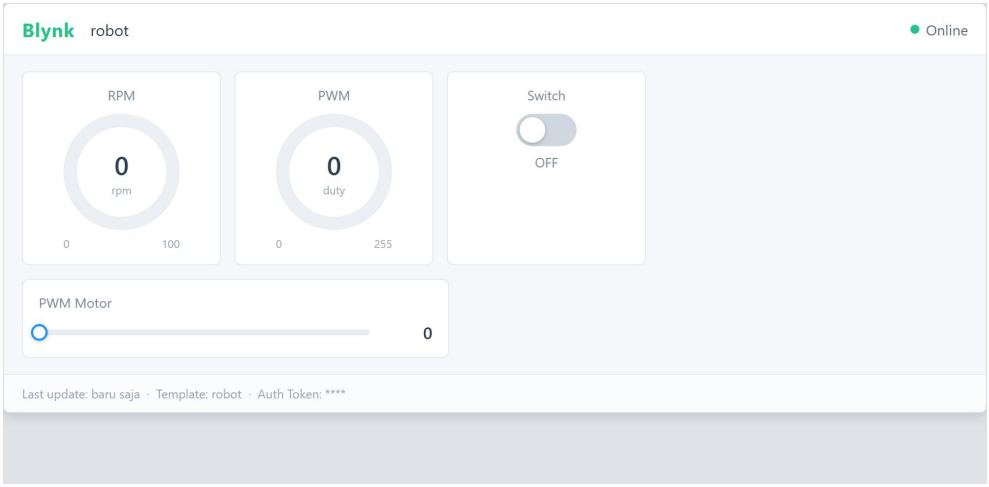
Pada pengujian ini dashboard menampilkan RPM 100 ketika motor berputar, sedangkan gauge PWM menunjukkan duty 200. Dari pembacaan kode terdapat empat catatan penting. Pertama, `hitungRPM()` melepaskan interrupt dengan `detachInterrupt()` lalu memasangnya kembali dengan `attachInterrupt()` setiap satu detik; cara ini memang melindungi pembacaan variabel `rev` dari gangguan, tetapi setiap pulsa yang tiba di celah antara detach dan attach akan hilang, sehingga pada putaran tinggi hasil pembacaan cenderung lebih rendah daripada nilai sebenarnya. Alternatif yang lebih baik adalah membaca `rev` di dalam blok kritis, yaitu `noInterrupts()` lalu `interrupts()`, sehingga interrupt tidak perlu dilepas sama sekali.

Kedua, `HOLES = 2` berarti piringan memiliki dua celah untuk setiap satu

putaran penuh; menetapkan nilai ini secara keliru akan membuat hasil RPM tepat dua kali lipat atau justru setengah dari nilai sebenarnya. Ketiga, debounce 500 mikrodetik membatasi laju pencacahan pulsa hingga 2000 pulsa per detik, sehingga kecepatan yang lebih tinggi dari batas itu tidak akan terukur dengan benar. Keempat, penulisan ganda ke virtual pin V0, V1, V2, dan V3 oleh `kirimKeBlynk()` setiap detik memang menjaga tampilan tetap mutakhir, tetapi menambah beban lalu lintas data ke server. Pembacaan RPM mentah tetap dapat diperiksa melalui Serial Monitor dengan perintah `R`, yang menampilkan nilai tanpa dibatasi skala datastream.

2.6 Keadaan dashboard setelah motor dimatikan

Pengujian ini bermaksud memeriksa apakah program sanggup mengembalikan seluruh tampilan dashboard ke titik nol, tanpa menyisakan widget yang masih memegang angka lama. Caranya adalah menekan sakelar dashboard ke posisi OFF.



Gambar 6. Keadaan dashboard setelah sakelar dimatikan

Seluruh angka pada dashboard kembali ke nol, yaitu RPM 0, PWM 0, dan slider 0. Perilaku

ini sesuai dengan `BLYNK_WRITE(V3)` yang menetapkan `dutyCycle = 0` ketika sakelar dimatikan, kemudian `applyMotor()` memadamkan LED indikator dan menghentikan motor. Nilai slider pun kembali ke nol karena `applyMotor()` juga menuliskan duty cycle terbaru ke V2, sehingga tidak ada widget yang tertinggal pada angka lama.

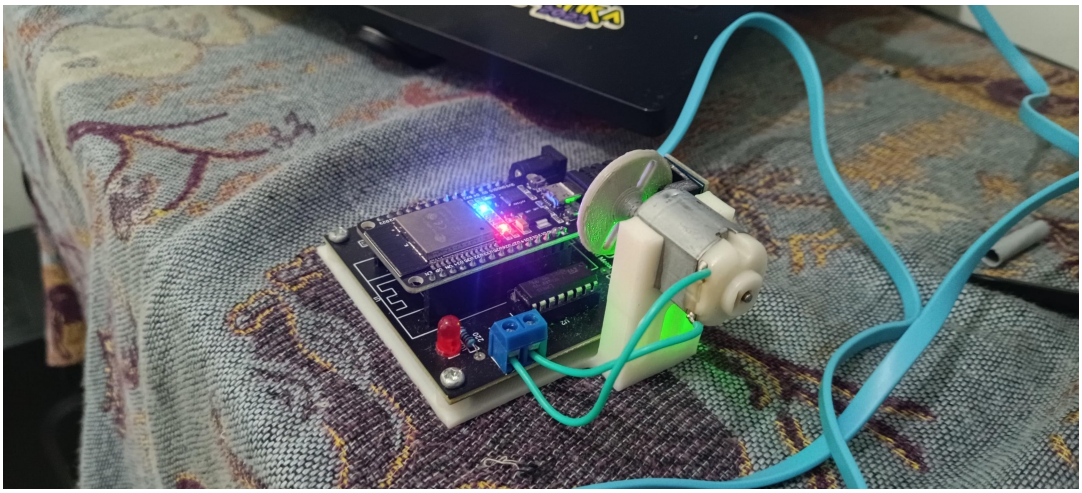
2.7 Rangkuman hasil pengujian

No	Aksi pada Dashboard	Hasil Pengamatan
1	Perangkat dinyalakan	Device berstatus online, PWM dan slider menampilkan 200
2	Sakelar ON	Motor berputar, LED biru menyala, PWM = 200
3	Slider digeser ke kanan	PWM naik menjadi 207, putaran motor bertambah cepat
4	Slider digeser ke kiri	PWM turun menjadi 195, putaran motor melambat
5	Sakelar OFF	RPM, PWM, dan slider menjadi 0, motor berhenti, LED padam
6	Gauge RPM	Tertahan pada 100 karena batas maksimum datastream V0 masih 100

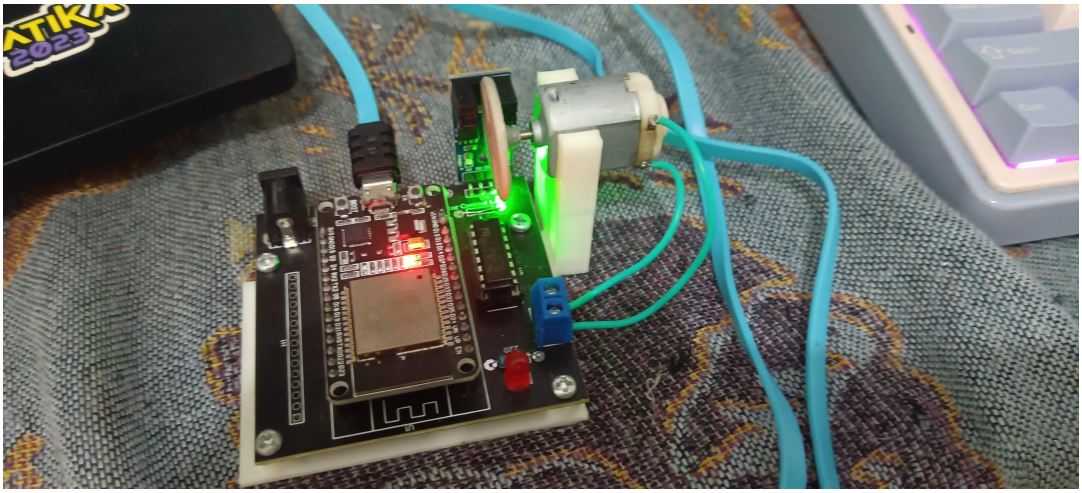
Seluruh aksi pada dashboard menghasilkan tanggapan yang sesuai pada perangkat keras, dan nilai antar widget senantiasa konsisten karena program menyinkronkan kedua arah setiap kali terjadi perubahan.

2.8 Dokumentasi Praktikum

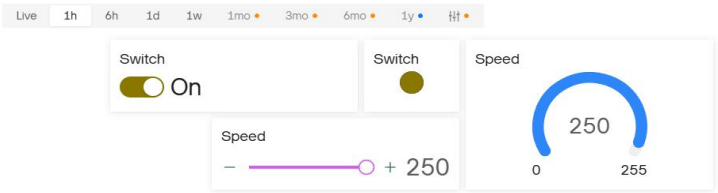
Praktikum modul ketiga memindahkan kendali motor dari kabel ke jaringan. Kit iMCLab dihubungkan ke WiFi, lalu sakelar dan slider pada dashboard Blynk dipakai untuk menyalakan motor serta mengatur duty cycle. Foto berikut mendokumentasikan keadaan kit beserta dashboard yang dipakai selama pengujian.



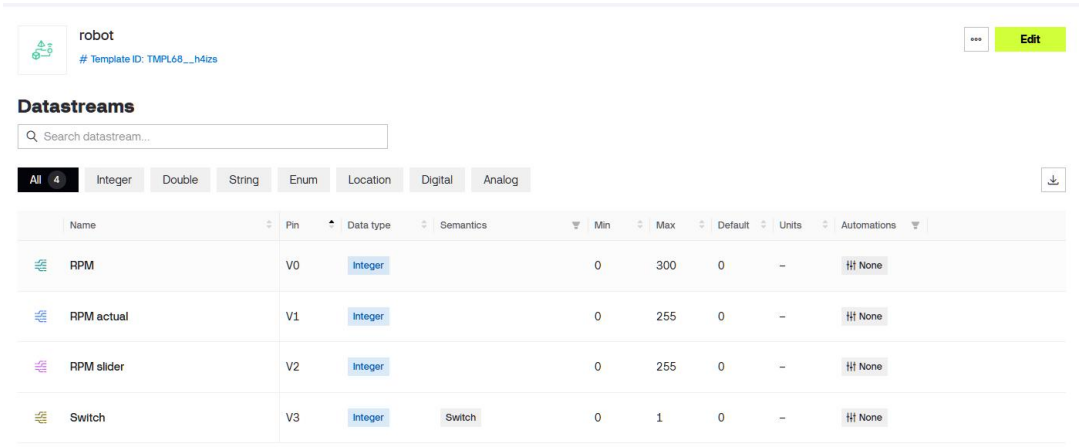
Gambar 7. Kit iMCLab saat kendali motor dipindahkan ke Blynk



Gambar 8. Kit iMCLab dan roda piringan saat dioperasikan dari dashboard Blynk



Gambar 9. Dashboard Blynk dengan sakelar, slider, dan gauge kecepatan



Gambar 10. Daftar datastream V0 sampai V3 pada Template Blynk

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan hasil pengujian, dapat disimpulkan beberapa hal berikut:

menggunakan Template ID dan token autentikasi, ditandai status online pada dashboard.

kecepatannya dapat diatur melalui slider (V2) pada rentang duty cycle 0–255.

`applyMotor()` dan `kirimKeBlynk()` menuliskan kembali nilai terbaru ke virtual pin.

putaran yang sesuai, sedangkan PWM 0 menghentikan motor dan memadamkan LED indikator.

tertahan pada angka 100 karena batas maksimum datastream V0 belum disesuaikan.

memerlukan kabel data ke komputer sehingga motor dapat dikendalikan dari mana saja selama perangkat terhubung internet.

3.2 Saran

Beberapa perbaikan yang dapat dipertimbangkan untuk pengembangan berikutnya:

`hitungRPM()` dengan blok kritis `noInterrupts()` dan `interrupts()` agar tidak ada pulsa yang hilang selama proses pembacaan `rev`.

sesungguhnya mungkin terjadi, sehingga gauge tidak lagi mentok pada angka 100.

agar lalu lintas data ke server Blynk lebih hemat.

khususnya pada perintah `+`, supaya status sakelar tidak pernah berbeda dari

keadaan fisik motor.

benar dipakai, karena kekeliruan satu angka akan menggandakan atau memotong hasil pembacaan RPM.